Modules

Modules are file containing Python statements and definitions.

A file containing python code is called a module
If the file is *my_lib.py*, the module name is *my_lib*

Modules are useful to break down large programs into small manageable and organized files.

Modules provide re-usability of code: useful functions can be put in a module and later imported in another module/script and re-used.

How to import module:
import my_module

How to import module by name:
import my_module as example

How to import a single function by a module:
from my_module import my_function

How to import all names in a module:
*from my_module import **

Example: module imports

File (module) my_libs.py:

```
def add(a, b):
   #This program adds two numbers and return the result
   result = a + b
   return result
def multiply(a,b)
   #This program multiply two numbers and return the result
   result = a * b
   return result
```

- Import the entire module (my_modules_example_1.py)

```
import my_libs
print “add 4 and 5.5. Result:”, my_libs.add(4,5.5)
```

- Import a single function (my_modules_example_2.py)

```
from my_libs import multiply
print "Multiply 4 X 5. Result:", multiply(4, 5)
```

- Import the entire module by name(my_modules_example_3.py)

```
import my_libs as example
print “add 4 and 5.5. Result:”, example.add(4,5.5)
```

Example: module os (manage OS dialog operations)

```
import os

#namespace of module os
>>> os.curdir
'.'
>>> os.getenv('HOME')
'/home/bertocco'
>>> os.listdir('.')
['my_modules_example_1.py',
 'read_by_line.py',
 '.mozilla',
 '.bash_logout',
.....
]
In [2]: import os

In [3]: os.defpath
Out[3]: ':/bin:/usr/bin'
```

Module Search Path

To import a module, Python looks at several places.
Interpreter first looks for a built-in module then the search is in this order:

- The current directory
- PYTHONPATH (an environment variable with a list of directory)
- The installation-dependent default directory

How to reload a Module

The Python interpreter imports a module only once during a session. This makes things more efficient.

If the module is changed during the course of the program, we would have to reload it. To reload the module you have two ways:

- Restart the interpreter (not much clean).

- Use the function reload() inside the imp module. Example:

In [29]: import my_libs

In [30]: import imp

In [31]: imp.reload(my_libs)
Out[31]: <module 'my_libs' from 'my_libs.pyc'>

Introspection

Introspection of a language is the ability of the language itself to provide information of its objects at runtime.

Python ha a very good support for introspection.

The interpreter can be used interactively to better know and understand the code.

Following the description of useful python built-in functions for introspection

dir()

The dir() function can be used to find out names that are defined inside a module.

For example, we have defined the functions add(a,b) and multiply(a,b) in the module my_libs. We can find them:

```
In [32]: dir(my_libs)
Out[32]:
['__builtins__',
 '__doc__',
 '__file__',
 '__name__',
 '__package__',
 'add',
 'multiply']
```

names that begin with an underscore are default Python attributes associated with the module (we did not define them ourself).

All the names defined in the current namespace can be found out using the dir() function without any arguments. Example (try):

```
dir()
```

help()

The function help is available for each module/object and allows to know the documentation for each function.
Try (in the interpreter) the commands:

```
import math
dir()
help(math.acos)
```

Example:

```
In [8]: import math
In [9]: dir(math)
Out[9]:
['__doc__',
 '__name__',
 '__package__',
 'acos',
 'acosh',
…….
In [19]: help(math.acos)
Help on built-in function acos in module math:
acos(...)
    acos(x)
    Return the arc cosine (measured in radians) of x.
```

type()

The type function allows to

```
In [1]: a=5
In [2]: type a
  File "<ipython-input-2-e92615c5fd0c>", line 1
    type a
         ^
SyntaxError: invalid syntax
In [3]: type(a)
Out[3]: int


In [5]: l = [1, "alfa", 0.9, (1, 2, 3)]
In [6]: print [type(i) for i in l]
[<type 'int'>, <type 'str'>, <type 'float'>, <type 'tuple'>]
```